Self-Falsifying Adaptive Systems: An Executable Falsification Workflow for AI-Assisted Research

Physis Research Program

September 24, 2026

Abstract

We describe an executable falsification workflow implemented in a production software system: claims are recorded with their evidence, experiments are run with construction-matched nulls, verdicts are assigned, revisions are tracked, and failed branches are retained as research evidence. The workflow was stress-tested over three months of autonomous AI-assisted research, producing both settled results (cross-embedder ontology agreement near chance) and a catalog of twelve failure modes where the null caught the error. We argue that the falsification workflow, not the grand architecture, is the transferable contribution.

1 Introduction

Autonomous research agents (Sakana AI Scientist, Robot Scientist, and others) typically follow a loop: generate hypotheses, run experiments, interpret results, update beliefs. The loop is usually described as hypothesis-driven science. What is less often described is the infrastructure that makes the loop honest: how are hypotheses recorded? How are experiments designed to be falsifiable? How are negative results preserved?

We report from a system that implements this infrastructure explicitly. Every claim has a ledger entry with a verdict (Supported, Contradicted, Partially Supported, Underpowered, Artifact, Superseded). Every experiment specifies a construction-matched null before running. Every correction is recorded and linked to the original claim. Failed branches are never deleted.

The system is Physis, a 200+ module Rust application for ontology management, coherence tracking, and agent coordination. Over three months of development, the system ran 90+ experiments across five research tracks. The experiments produced settled results — including a demonstration that cross-embedder ontology agreement is near chance (ARI 0.10) and that cross-agent failure priors transfer with a coverage gate — and a dozen failure modes where the null caught the error.

2 Architecture

The falsification workflow has five stages:

1. **Claim registration.** A hypothesis is recorded with its predicted effect, the experiment that will test it, and the null that could kill it. The claim has a unique ID and a timestamp.

2. **Experiment execution.** The experiment is run. The null is run in the same execution, on the same data, with the same code path. The null is not an afterthought; it is a required input.

3. **Verdict assignment.** The system assigns a verdict: Supported (effect size excludes the null), Contradicted (the null explains the result), Partially Supported (the effect survives some controls but not others), Underpowered (the experiment cannot resolve the question), Artifact (the result is explained by a confound), or Superseded (a later experiment modifies the claim).

4. **Revision tracking.** When a claim is revised, the original is marked Superseded, not deleted. The revision chain is preserved: claim -¿ experiment -¿ verdict -¿ revision -¿ new claim.

5. **Failed branch retention.** Experiments that produce null results are retained in the repository. Failed code is marked deprecated, not deleted. The experimental trail is never erased.

3 Stress Test: What the Workflow Caught

The twelve failure modes cataloged in the companion paper (Paper 5) were all caught by the workflow:

- **E14 (wrong injection distribution).** The claim "geometry scores misfilings at AUC 0.880" was registered with a null: injection at the nearest alternative cell. The null produced AUC 0.506, at chance. The claim was marked Contradicted. Without the pre-registered null, the 0.880 would have been reported as a positive.

- **E83 (generated fixture circularity).** The claim "the antonym rule detects contradiction" was registered with a null: hand-written fixtures without lexical antonyms. The null produced recall 0.143. The claim was marked Artifact.

- **E85 (adversarial collapse).** The claim "structured rules achieve precision 1.000" was registered with a null: adversarial fixtures designed to fire each rule wrongly. The null produced precision 0.000. The claim was marked Contradicted.

- **E17 (confound check that stops early).** The claim "ANCHORVOC is not circular" was registered with three tests. The workflow required all three to pass. Test 3 failed, and the claim was marked Artifact, even though Tests 1 and 2 passed. Without the requirement to test all three, the script's first verdict (which read only Test 1) would have been accepted.

- **E10 (pre-flight overridden).** The power analysis returned STOP, and the run proceeded anyway. The workflow records this as a violation of the pre-registration contract. The claim is marked Underpowered.

In each case, the workflow caught the error because the null was pre-registered, the baseline was rerun in the same execution, and the verdict was assigned by the system, not by the agent.

4 Comparison with Existing Systems

Sakana AI Scientist (2024, 2025) implements an autonomous research loop: generate hypotheses, run experiments, interpret results, update beliefs. The loop is hypothesis-driven, but the falsification infrastructure (how hypotheses are recorded, how experiments are designed to be falsifiable, how negative results are preserved) is not described in detail. Our contribution is the infrastructure, not the loop.

AGM belief revision (Alchourron, Gardenfors & Makinson, 1985) provides a formal theory of belief change: contraction, expansion, revision. Our workflow is an implementation of these principles in a running system, with the addition of construction-matched nulls and machine-readable experiment histories.

Doyle's TMS (1979) is a truth maintenance system: beliefs are justified, and a belief lives while a justification holds. Our workflow is similar in spirit but operates on empirical claims (with nulls and effect sizes) rather than logical propositions.

Robot Scientist (King et al., 2009) generates hypotheses, runs experiments, and interprets results. The Robot Scientist's hypotheses are formalized in logic; our claims are formalized as hypothesis + null + verdict + revision chain.

The distinctive feature of our workflow is the **retained failed branch**: the experiment ledger preserves not just the surviving claims but the full chain of claims that were tested and failed. This makes the experimental trail reproducible and auditable.

5 Limitations

Single system, single domain. The workflow was tested on one system (Physis) in one domain (ontology management and agent coordination). Its generalizability to other domains is untested.

No comparison group. We cannot claim that the workflow produces better science than a workflow without it. Such a comparison would require a controlled study with two research groups using different workflows.

Agent compliance. The workflow catches errors only if the agent follows it. E10 (pre-flight overridden) shows that an agent can bypass the workflow. The workflow records the violation but cannot prevent it.

Scale. The workflow was tested on 90+ experiments over three months. Its performance at larger scales (hundreds of agents, thousands of experiments) is unknown.

6 Conclusion

The falsification workflow — claim registration, pre-registered nulls, in-run baselines, revision tracking, failed branch retention — is a transferable contribution. It is not tied to the Physis architecture or to any particular research domain. Any system that uses AI agents to design, implement, and interpret experiments can adopt it.

The twelve failure modes cataloged in the companion paper demonstrate that the workflow is not ceremonial: in each case, the null caught an error that would otherwise have been reported as a positive result.